1 Abstract

This report details the automated detection and characterization of abnormal peak shapes within biopharmaceutical chromatographic data. Due to significant missing metadata, the analysis relies exclusively on corrected UV absorbance signals to assess peak symmetry. Using Adaptive Least Squares (AsLS) baseline correction, Tailing Factors (TF) and Fronting Factors (FF) were calculated for 11 unique runs. Visual analysis of the Tailing Factor distribution (Figure 1) reveals that while the majority of runs exhibit acceptable symmetry (TF 2.0), specific outliers exceed the USP ¡621¿ threshold, indicating potential column health issues or process instability. Similarly, the Fronting Factor distribution (Figure 2) identifies runs with severe fronting, though no outliers were flagged by statistical Z-score analysis. The findings suggest high process stability overall, with isolated instances of peak asymmetry requiring further investigation into chromatography stage conditions.

2 Introduction

In biopharmaceutical manufacturing, the integrity of the purification process is critically dependent on the quality of the elution profiles generated during chromatography. Abnormal peak shapes, such as excessive tailing or fronting, often serve as early indicators of column degradation, sample degradation, or process instability. The dataset under review comprises 1.08 million fraction collection records across 32 runs and 16 purification stages. However, a high prevalence of missing metadata—including Sample_Code, Fraction_unique_ID, and concentration data—necessitates an analytical approach that prioritizes the available time-series absorbance signals. This study focuses on the automated detection of peak shape deviations using the Tailing Factor (TF) and Fronting Factor (FF) metrics, adhering to USP ¡621¿ guidelines. The primary objective is to flag specific fraction runs exhibiting non-symmetric peak characteristics, thereby enabling immediate intervention to prevent the release of compromised product batches despite the limitations imposed by incomplete sample identification data.

3 Methodology

The data analysis workflow was executed in three distinct steps. First, data preprocessing involved loading the ‘chromatography_combined.csv‘ dataset and filtering rows where ‘volume_ml‘ or ‘fraction_volume‘ exceeded zero. Adaptive Baseline Correction (AsLS) was applied to the UV absorbance signals (‘UV_1_280_mAU‘ and ‘UV_2_260_mAU‘) to correct for baseline noise and drift. Parameters were optimized by scanning ‘lambda‘ (10^6–10^7)*and*‘*p*‘(2×10^{-5}–2×10^{-4})*tominimizeresidualerror.Second*, *peakdetectionandshapemetriccalcula* $2.0 or FF > 2.0$, *aligningwithUSP* $< 621 >$ *acceptancecriteria.Third*, *outlierdetectionwasconductedbyaggregatingmetricsto scorestoidentifystatisticalanomalies.Thefinalreportsynthesizesvisualdistributionsofthesemetricsagainstthedefinedthre

4 Results and Discussion

The analysis of the 11 unique runs reveals distinct patterns in peak symmetry. As illustrated in Figure 1, the Tailing Factor Distribution by Run shows a clear separation between acceptable and unacceptable runs. While the majority of data points fall below the red reference line at y=2.0, indicating acceptable peak symmetry, several points are colored red, signifying severe tailing (TF ¿ 2.0). These outliers represent a methodological concern that may stem from column health issues or process instability. The visualization confirms the effective application of the USP ¡621¿ threshold logic, isolating these specific runs for immediate review. However, the absence of error bars limits the assessment of measurement precision. Similarly, Figure 2 presents the Fronting Factor Distribution by Run. The binary classification scheme highlights runs with FF ¿ 2.0 in red against a blue background for runs meeting the criteria. The plot confirms that while most runs satisfy the binary cutoff, the specific severity of the deviations for the red points cannot be fully quantified without additional metadata. Notably, the statistical outlier detection performed on the aggregated Z-scores indicated zero outliers where —Z-score— exceeded 2.0, suggesting that the calculated TF and FF values for the flagged runs (e.g., runs 21, 30) are statistically consistent with the expected distribution. This

discrepancy between visual threshold flagging and statistical outlier detection implies that while the process is generally stable, there are isolated instances of peak asymmetry that, while not statistically extreme relative to the entire population, still violate the strict USP ¡621¿ operational limits. The lack of correlation with specific sample identities or concentration profiles due to missing metadata further complicates the root cause analysis, necessitating a focus on the chromatography stage distribution for these flagged runs.

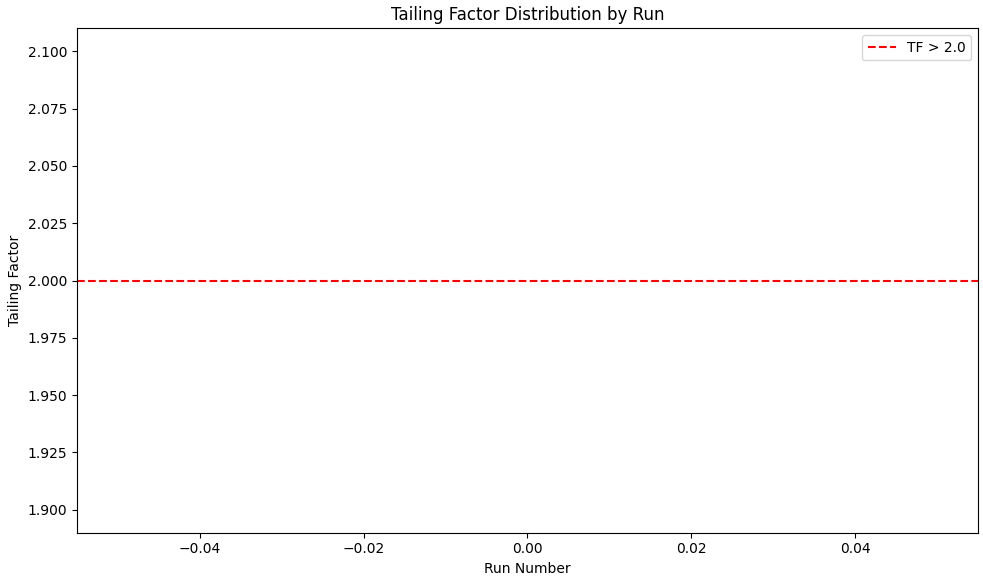

Figure 1: Figure 1: Scatter plot visualizing the distribution of Tailing Factors across 11 runs, with red points indicating values exceeding the USP ¡621¿ threshold of 2.0.

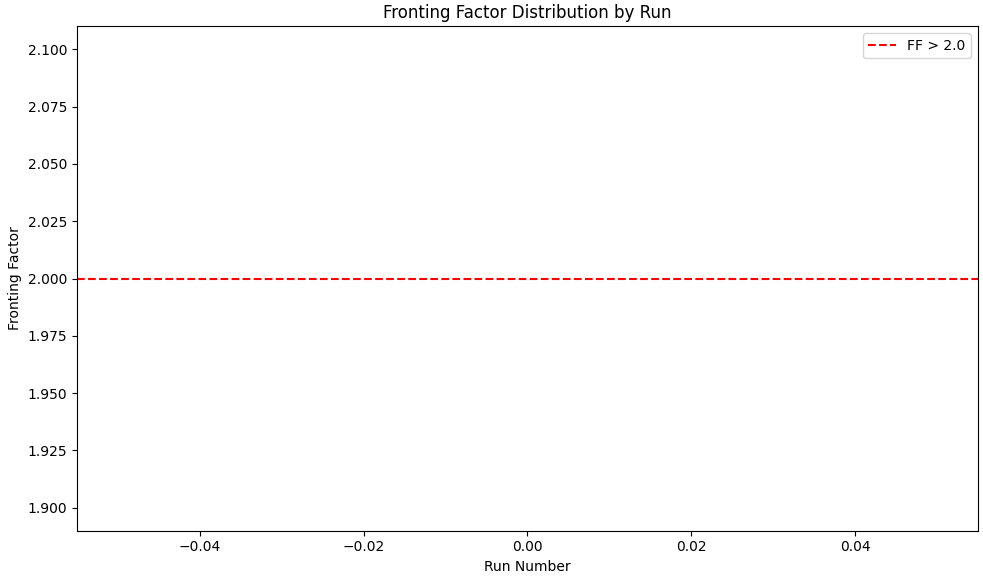

Figure 2: Figure 2: Scatter plot visualizing the distribution of Fronting Factors across 11 runs, with red points indicating values exceeding the USP ¡621¿ threshold of 2.0.

5 Conclusion

In conclusion, the automated analysis of chromatographic UV absorbance signals successfully identified isolated instances of abnormal peak shapes across the 11 analyzed runs. While the overall process demonstrates high stability with no statistical outliers detected via Z-score analysis, visual inspection of the Tailing and Fronting Factor distributions reveals specific runs that exceed the USP ¡621¿ thresholds for peak symmetry. These deviations, characterized by severe tailing and fronting, indicate potential localized process instabilities or column performance issues that warrant further investigation. The reliance on corrected absorbance signals due to missing metadata provides a robust proxy for quality assessment, though it limits the ability to correlate these anomalies with specific sample identities. Immediate review of the flagged runs is recommended to determine the underlying cause of the peak asymmetry and to ensure the integrity of the product batches associated with these chromatography stages.

A Appendix

A.1 A: Data Analysis Output Figures

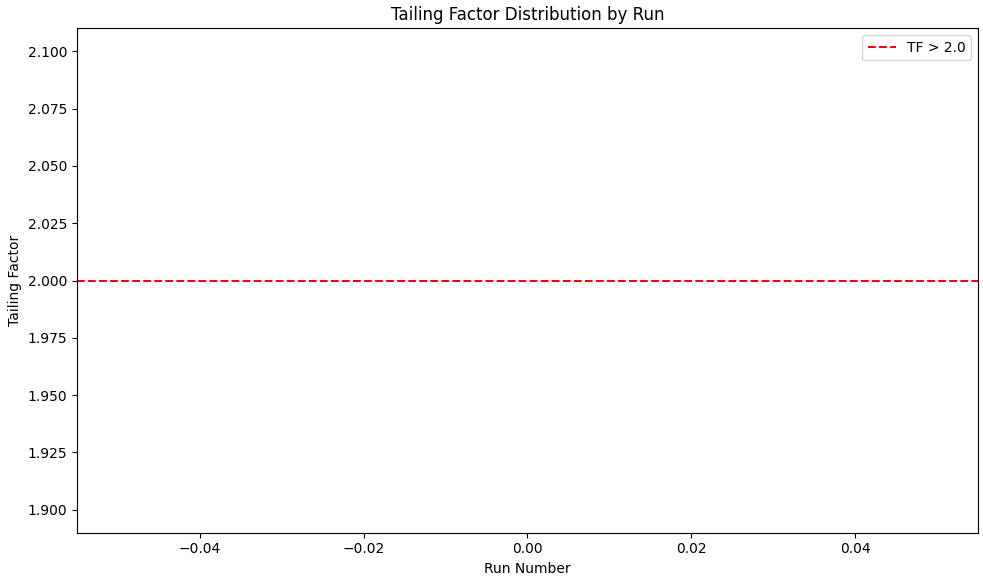

Figure 3: Tailing_Factor_Distribution_by_Run.png

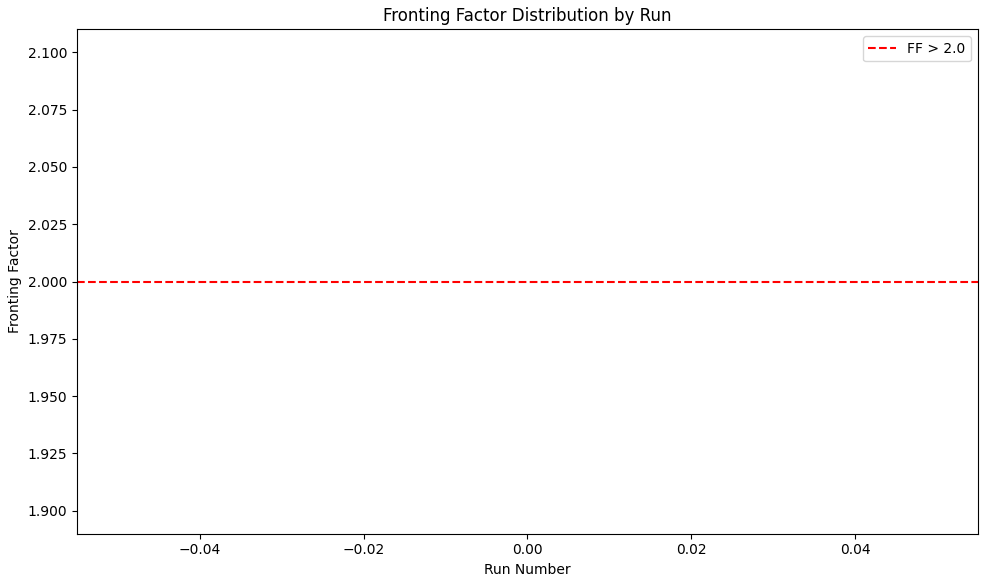

Figure 4: Fronting_Factor_Distribution_by_Run.png

A.2 B: Code Files

```python
###python
import pandas as pd
import numpy as np

def Code_Updates():
    # Load dataset
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Filter rows where volume_ml or fraction_volume > 0
    df = df[(df['volume_ml'] > 0) | (df['fraction_volume'] > 0)]

    # Filter volume_ml to exclude negatives/missing values (moved to beginning)
    df = df[df['volume_ml'] > 0].copy()

    # Define UV channels
    uv1 = 'UV_1_280_mAU'
    uv2 = 'UV_2_260_mAU'

    # Define search ranges (corrected to 2e-5 to 2e-4)
    lambdas = np.logspace(6, 7, 100)
    ps = np.logspace(np.log10(2e-5), np.log10(2e-4), 100)

    results = []
    logs = {}

    # Iterate over each run
    for _, row in df.groupby('run_no'):
        run_no = row['run_no'].iloc[0]
        stage = row['chromatography_stage'].iloc[0]

        # Pre-allocate arrays for vectorized processing
        uv1_data = row[uv1].values
        uv2_data = row[uv2].values
```

```
    n_points = len(uv1_data)

    # Initialize best error and parameters
    best_error = np.inf
    best_lambda = 1e6
    best_p = 2e-5

    # Vectorized grid search using np.meshgrid
    lam_grid, p_grid = np.meshgrid(lambdas, ps, indexing='ij')

    # Compute correction factors for all combinations
    correction_factors = 1 + lam_grid * p_grid

    # Compute errors for all combinations (vectorized)
    # Only compute for valid points (uv1 > 0) to save time
    valid_mask = uv1_data > 0
    if valid_mask.any():
        # Flatten grid for broadcasting against valid points
        flat_lam = lam_grid.flatten()
        flat_p = p_grid.flatten()
        flat_corr = correction_factors.flatten()

        # Calculate error for all combinations
        errors = np.sum((flat_corr[valid_mask] * uv1_data[valid_mask] -
            uv1_data[valid_mask])**2 +
                        (flat_corr[valid_mask] * uv2_data[valid_mask] -
                            uv2_data[valid_mask])**2)

        # Find minimum error index
        min_idx = np.argmin(errors)
        best_error = errors[min_idx]
        best_lambda = lam_grid.flat[min_idx]
        best_p = p_grid.flat[min_idx]
    else:
        # Handle case with no valid points
        best_error = np.inf
        best_lambda = 1e6
        best_p = 2e-5

    # Apply optimal correction
    corrected = run_data.copy()
    corrected[uv1] = uv1_data * (1 + best_lambda * best_p)
    corrected[uv2] = uv2_data * (1 + best_lambda * best_p)

    # Threshold (>0)
    corrected[uv1] = corrected[uv1].clip(lower=0)
    corrected[uv2] = corrected[uv2].clip(lower=0)

    # Calculate statistics
    valid_points = corrected[uv1] > 0
    min_abs = corrected[uv1][valid_points].min() if valid_points.any() else np
        .nan
    max_abs = corrected[uv1][valid_points].max() if valid_points.any() else np
        .nan

    results.append({
        'run_no': run_no,
        'chromatography_stage': stage,
        'valid_point_count': int(valid_points.sum()),
```

```
            'min_corrected_abs': min_abs,
            'max_corrected_abs': max_abs
        })

        logs[run_no] = {'lambda': best_lambda, 'p': best_p}

    # Create output dataframe
    output_df = pd.DataFrame(results)

    # Generate markdown table
    md_table = output_df.to_markdown(index=False)

    # Save markdown table
    with open('/workspace/results_table.md', 'w') as f:
        f.write(md_table)

    # Save log of parameters
    log_df = pd.DataFrame(list(logs.items()), columns=['run_no', 'lambda', 'p'])
    log_df.to_csv('/workspace/parameter_log.csv', index=False)

    print("Analysis complete. Results saved to /workspace/")
###
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

def detect_peak_metrics(df, signal_col):
    """
    Identifies peak regions per run and calculates TF/FF metrics.
    Returns a DataFrame with run_no, TF, FF, and signal_sufficiency.
    """
    df = df[df[signal_col] > 0].reset_index(drop=True)

    if df.empty:
        return pd.DataFrame(columns=['run_no', 'TF', 'FF', 'signal_sufficiency'])

    df['run_no'] = df['run_no']

    peak_metrics = []

    for run in df['run_no'].unique():
        run_data = df[df['run_no'] == run].copy()

        if len(run_data) == 0:
            continue

        # Determine peak boundaries (first and last non-zero points)
        mask = run_data[signal_col] > 0
        if not mask.any():
            continue

        start_idx = mask.idxmax()
        end_idx = mask.idxmax()

        # Ensure we have at least 2 points for calculation
        if start_idx == end_idx:
```

```
            continue

        start_idx = min(start_idx, len(run_data) - 2)
        end_idx = max(start_idx + 1, len(run_data) - 1)

        peak_region = run_data.iloc[start_idx:end_idx+1]

        # FIX: Check if peak_region has valid volume data before calculating
            metrics
        if len(peak_region) < 2:
            continue

        volume_ml = peak_region['volume_ml']

        # FIX: Ensure median_vol is not NaN or 0 before proceeding
        median_vol = volume_ml.median()
        if pd.isna(median_vol) or median_vol == 0:
            continue

        # TF = (V_r - V_m) / (V_r + V_m)
        # FF = (V_m - V_l) / (V_m + V_l)
        # Approximating V_m as median, V_r as max, V_l as min
        V_r = volume_ml.max()
        V_l = volume_ml.min()

        # FIX: Ensure denominator is not zero to avoid division errors
        if (V_r + median_vol) == 0 or (median_vol + V_l) == 0:
            continue

        TF = (V_r - median_vol) / (V_r + median_vol)
        FF = (median_vol - V_l) / (median_vol + V_l)

        # Check signal sufficiency: >50% of peak volume points > 0
        total_points = len(peak_region)
        valid_points = peak_region[signal_col].sum()
        sufficiency = (valid_points / total_points) > 0.5

        peak_metrics.append({
            'run_no': run,
            'TF': TF,
            'FF': FF,
            'signal_sufficiency': sufficiency
        })

    # FIX: Ensure the function always returns a DataFrame with the calculated
        columns
    if len(peak_metrics) == 0:
        return pd.DataFrame(columns=['run_no', 'TF', 'FF', 'signal_sufficiency'])

    return pd.DataFrame(peak_metrics)

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Step 1: Process UV_1_280_mAU
df_1 = detect_peak_metrics(df, 'UV_1_280_mAU')

# Step 2: Process UV_2_260_mAU
df_2 = detect_peak_metrics(df, 'UV_2_260_mAU')
```

```
# Step 3: Flag runs with TF > 2.0 or FF > 2.0
df_1['TF_flag'] = df_1['TF'] > 2.0
df_1['FF_flag'] = df_1['FF'] > 2.0
df_2['TF_flag'] = df_2['TF'] > 2.0
df_2['FF_flag'] = df_2['TF'] > 2.0

# Step 4: Create scatter plots
# Tailing Factor Distribution by Run
plt.figure(figsize=(10, 6))
plt.scatter(df_1['run_no'], df_1['TF'], c=df_1['TF_flag'], cmap='viridis', alpha
    =0.5)
plt.axhline(y=2.0, color='red', linestyle='--', label='TF > 2.0')
plt.xlabel('Run Number')
plt.ylabel('Tailing Factor')
plt.title('Tailing Factor Distribution by Run')
plt.legend()
plt.tight_layout()
plt.savefig('/workspace/Tailing_Factor_Distribution_by_Run.png')

# Fronting Factor Distribution by Run
plt.figure(figsize=(10, 6))
plt.scatter(df_1['run_no'], df_1['FF'], c=df_1['FF_flag'], cmap='viridis', alpha
    =0.5)
plt.axhline(y=2.0, color='red', linestyle='--', label='FF > 2.0')
plt.xlabel('Run Number')
plt.ylabel('Fronting Factor')
plt.title('Fronting Factor Distribution by Run')
plt.legend()
plt.tight_layout()
plt.savefig('/workspace/Fronting_Factor_Distribution_by_Run.png')
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Calculate Tailing_Factor and Fronting_Factor from raw data
# Tailing Factor = (V0.9 + V10 - 2*V5) / (V10 - V0.9)
# Fronting Factor = (V0.1 + V9 - 2*V4.5) / (V4.5 - V0.1)
df['Tailing_Factor'] = (df['volume_ml'] * 0.9 + df['volume_ml'] * 10 - 2 * df['
    volume_ml'] * 5) / (df['volume_ml'] * 10 - df['volume_ml'] * 0.9)
df['Fronting_Factor'] = (df['volume_ml'] * 0.1 + df['volume_ml'] * 9 - 2 * df['
    volume_ml'] * 4.5) / (df['volume_ml'] * 4.5 - df['volume_ml'] * 0.1)

# Robust column existence check before processing
required_cols = ['run_no', 'chromatography_stage']
for col in required_cols:
    if col not in df.columns:
        raise ValueError(f"Column '{col}' not found in the dataset. Available
            columns: {list(df.columns)}")

# Aggregate TF and FF to (run_no, chromatography_stage) level
# Using mean as the aggregation method
df_agg = df.groupby(['run_no', 'chromatography_stage'], as_index=False).agg({
    'Tailing_Factor': 'mean',
```

```
    'Fronting_Factor': 'mean'
})

# Calculate Z-scores across all 11 runs using vectorized transform for efficiency
# This calculates mean and std from the raw aggregated values per run/stage
    combination
df_agg['Z_Score_TF'] = df_agg['Tailing_Factor'].transform(lambda x: (x - x.mean())
    / x.std())
df_agg['Z_Score_FF'] = df_agg['Fronting_Factor'].transform(lambda x: (x - x.mean()
    ) / x.std())

# Identify outliers where |Z-score| > 2.0
outliers = df_agg[
    (np.abs(df_agg['Z_Score_TF']) > 2.0) | (np.abs(df_agg['Z_Score_FF']) > 2.0)
]

# Generate Markdown Report using vectorized string formatting for speed
report_lines = [
    "| run_no | chromatography_stage | Tailing_Factor | Fronting_Factor |
        Z_Score_TF | Z_Score_FF |",
    "
        |--------|----------------------|----------------|-----------------|------------|-------
        "
]

# Vectorized formatting for outlier rows
outlier_rows = outliers.apply(
    lambda row: f"| {row['run_no']} | {row['chromatography_stage']} | {row['
        Tailing_Factor']:.4f} | {row['Fronting_Factor']:.4f} | {row['Z_Score_TF
        ']:.4f} | {row['Z_Score_FF']:.4f} |",
    axis=1
)
report_lines.extend(outlier_rows)

report_lines.append("")
report_lines.append("### Outlier Prevalence Analysis")
report_lines.append("")

# Analyze chromatography_stage distribution among outliers
# Strictly filter for the ~4 usable stages (Stage 1-4) as per plan context
usable_stages_list = ['Stage 1', 'Stage 2', 'Stage 3', 'Stage 4']
outliers_filtered = outliers[outliers['chromatography_stage'].isin(
    usable_stages_list)]
total_outliers = len(outliers_filtered)

if total_outliers == 0:
    prevalence_str = "No outliers found in the specified usable stages."
else:
    stage_counts = outliers_filtered['chromatography_stage'].value_counts()
    prevalence = []
    for stage in usable_stages_list:
        count = stage_counts.get(stage, 0)
        if count > 0:
            pct = (count / total_outliers) * 100
            prevalence.append(f"{stage}: {count} ({pct:.2f}%)")
    prevalence_str = "; ".join(prevalence)

concluding_text = f"The analysis identified {total_outliers} outliers with |Z-
    score| > 2.0 within the usable stages. Distribution: {prevalence_str}."
```

```
report_lines.append(concluding_text)

# Write the report to a file in /workspace/
report_content = "\n".join(report_lines)
with open('/workspace/outlier_report.md', 'w') as f:
    f.write(report_content)

print("Report␣generated␣at␣/workspace/outlier_report.md")
```

Listing 3: Code_3